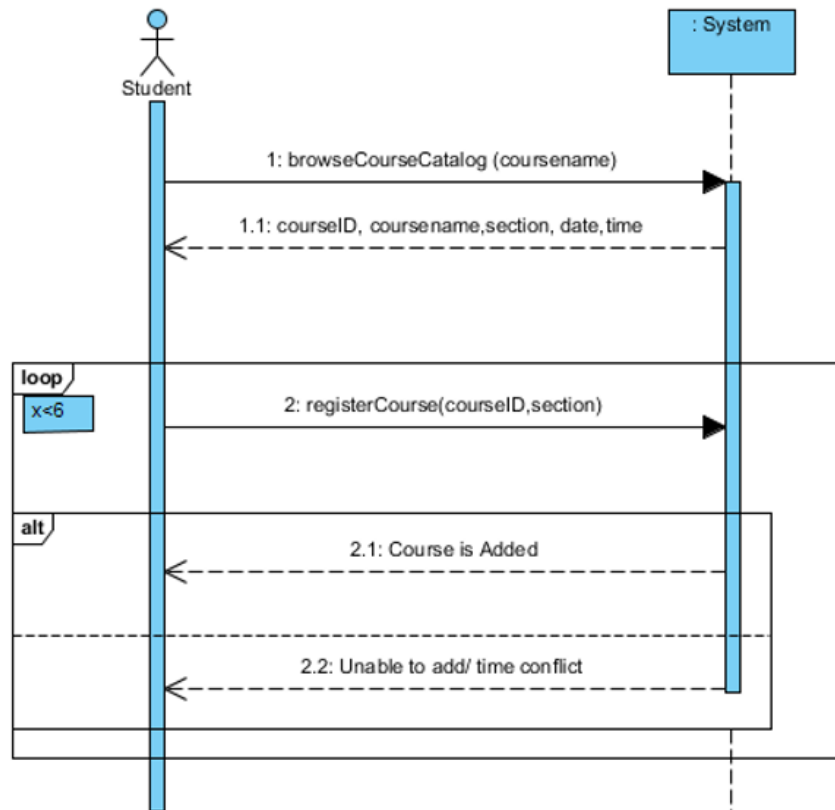
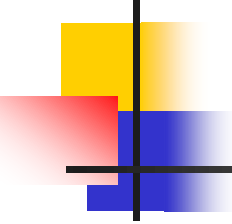


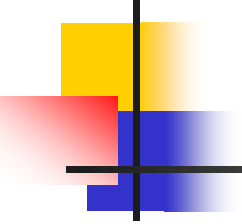
System Sequence Diagrams





System Sequence Diagrams

- Use cases describe how external actors interact with the software system. During this interaction, an actor generates events.
- A system sequence diagram (SSD) is a fast and easily created artifact that illustrates input and output events related to the system.
- SSDs are input to operation contracts and object design.



Motivation

What events are coming in to our system? **Why?**

- We have to design the software to handle these events and execute a response

- **Basically, a software system reacts to three things**

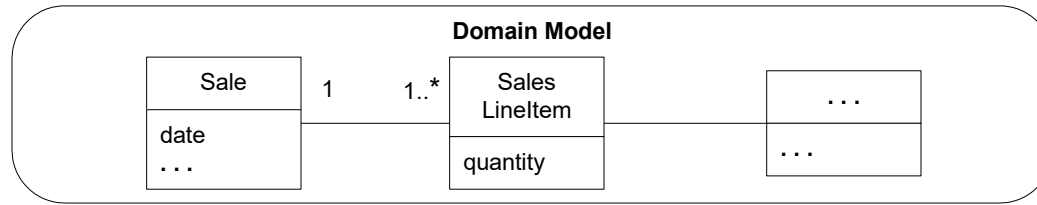
1. external events from actors (humans or computers)
2. timer events
3. faults or exceptions (which are often from external sources)

Important part of analyzing system behavior

- System behavior is a description of **what** a system does, **without explaining how** it does it.
- One part of that description is a system sequence diagram.
- Other parts include the use cases and system operation contracts (to be discussed later).

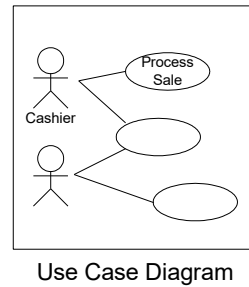
Sample UP Artifact Relationships

Business Modeling

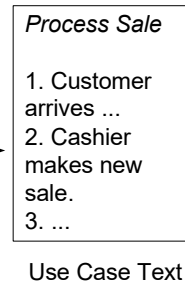


Use-Case Model

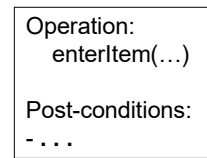
Requirements



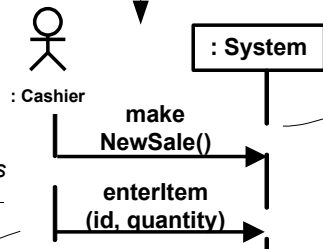
use case names



system events



system operations



Operation Contracts

System Sequence Diagrams

Vision



Glossary

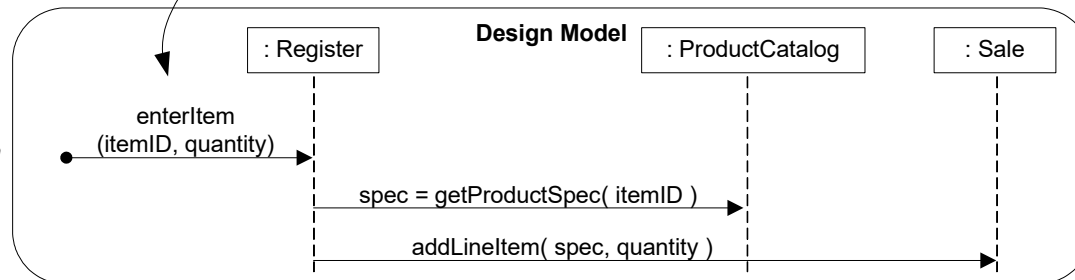


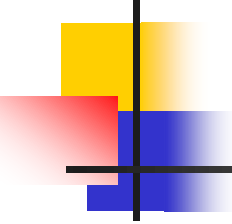
Supplementary Specification



starting events to design for

Design





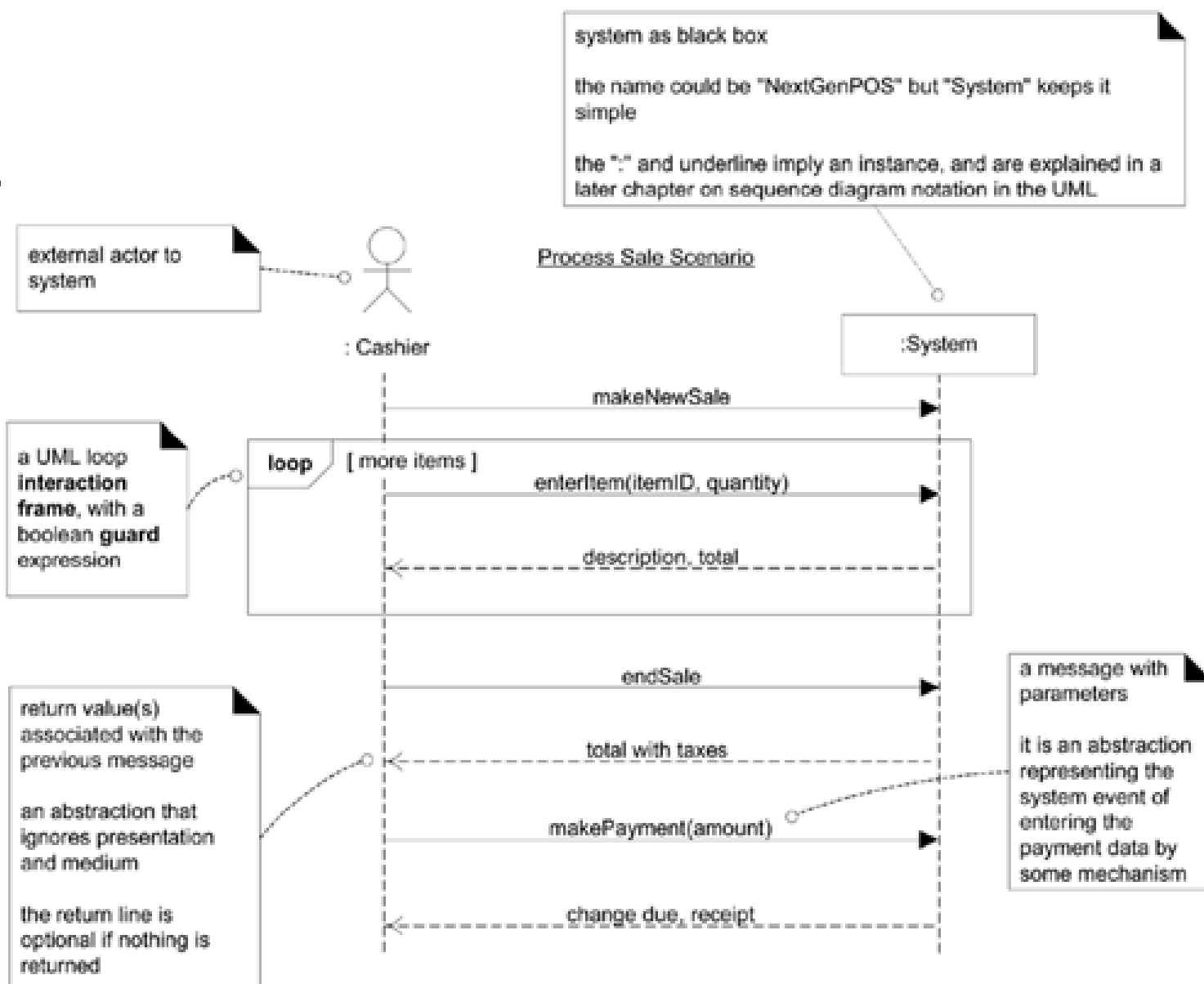
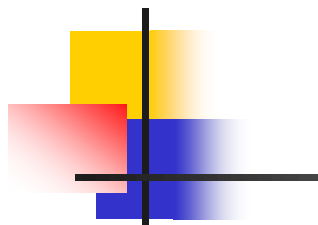
System Sequence Diagrams (SSDs)

A sequence diagram is a picture that shows, for a particular scenario of a use case, the events that external actors generate

Time proceeds downward, and the ordering of events should follow their order in the scenario

All systems are treated as a black box

Emphasis on events that cross the system boundary from actors to system

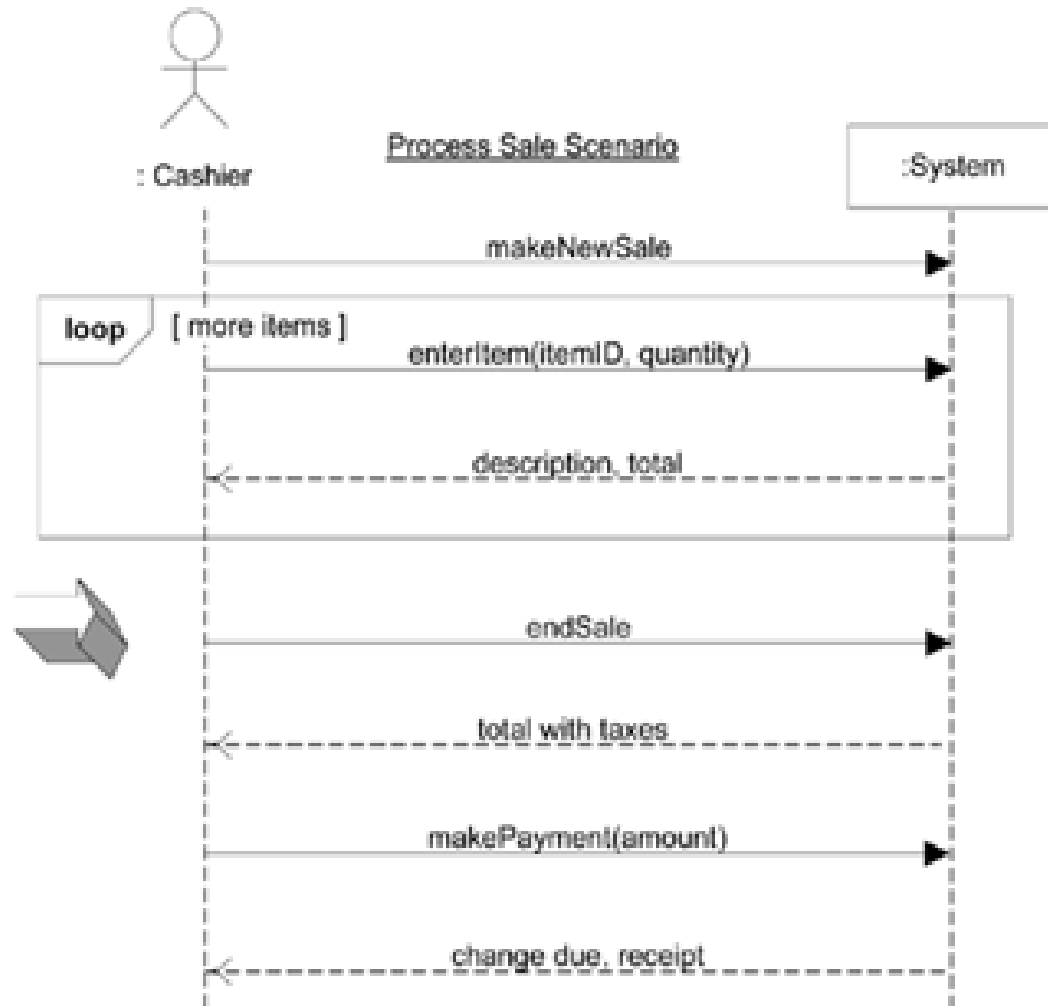


Relationship Between SSDs and Use Cases

Simple cash-only Process Sale scenario:

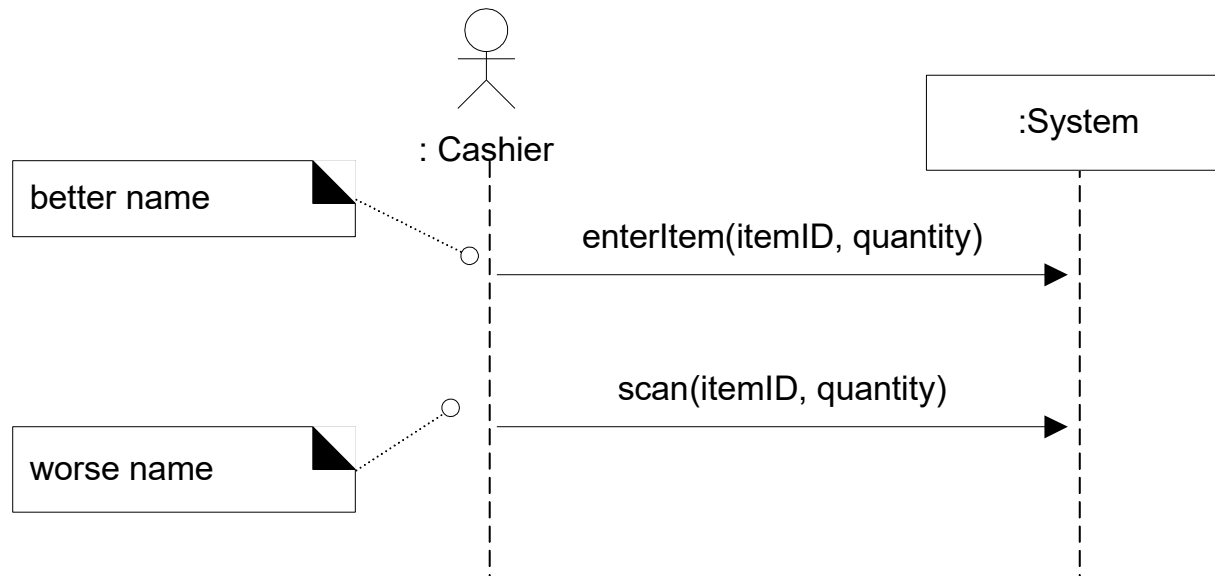
1. Customer arrives at a POS checkout with goods and/or services to purchase.
2. Cashier starts a new sale.
3. Cashier enters item identifier.
4. System records sale line item and presents item description, price, and running total.
Cashier repeats steps 3-4 until indicates done.
5. System presents total with taxes calculated.
6. Cashier tells Customer the total, and asks for payment.
7. Customer pays and System handles payment.

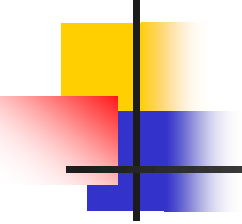
...



Naming System Events and Operations

- The set of all required system operations is determined by identifying the system events.
 - makeNewSale()
 - enterItem(itemID, quantity)
 - endSale()
 - makePayment(amount)





Conclusions

- We can draw an SSD for a main success scenario of each use case, and frequent or complex alternative scenarios.
- SSDs help to validate, clarify and refine use cases.
- Don't create SSDs for all scenarios, rather draw them only for the scenarios chosen for the next iteration.
- SSDs represent visually the essential aspects of a use case.
- For each event that the system receives from an actor, the system is expected to implement an operation, of the same name, to perform something of value to the actors.